

Lesson 10 · Jev and System One models

PDF: [this lesson](#) · [slide deck](#) · **Install (Linux · macOS · Windows):** [guide](#)

Part of [local-ai-lab](#) - a hands-on course for building local AI.

Interactive version (slides):

<https://nikolareljin.github.io/local-ai-lab/lesson-10-jev-system-one.html> **Read it locally (no GitHub Pages):** `./run -l 10 lesson` **Course home:** <https://nikolareljin.github.io/local-ai-lab/>
Source: <https://github.com/nikolareljin/local-ai-lab> **Author:** [Nik Reljin](#) **Time:** ~75 min ·
Prerequisites: Lesson 1 (Lessons 4, 5 and 9 helpful) · full objectives in [SYLLABUS.md](#)

Lessons: 1 · [RAG](#) → 2 · [MCP](#) → 3 · [Hybrid retrieval](#) → 4 · [RAG safety](#) → 5 · [RAG evaluation](#) → 6 · [Repo assistant](#) → 7 · [LangChain](#) → 8 · [LangGraph](#) → 9 · [Ollama tools](#) → **10 · Jev / System One (you are here)** → 11 · [Semantic Kernel](#) → 12 · [Bedrock Agents](#) → 13 · [Google ADK](#) → 14 · [AI-assisted testing](#) → 15 · [AI code review](#) → 16 · [Docs from changes](#)

Status: working demo, special lesson. Runnable in **Python, Node.js and C#**. The demo installs nothing and needs no model: it replays answers recorded from real runs. The local session needs Ollama and one small model; the one real call to Jev needs a TypeSafe API key.

The problem: 4,000 calls a day, six decisions each

A claims line at an insurer takes a call. Before a human does anything useful, somebody has to decide a few boring things:

- Which **line** of insurance is this? (auto, home, health, travel, life)
- What does the caller **want**? (a new claim, a status, a coverage question, to complain, to cancel)
- How **serious** is it?
- Does someone need **help right now**?
- Does the story **smell**? (no police report, policy bought last Tuesday, "can you pay cash today")
- Does a **human adjuster** need to look at it?

Six small decisions, 4,000 times a day. None of them needs an essay. All of them need to be **right, fast, cheap, and one of a fixed list of answers**, because the next thing in line is not a person. It is an `if` statement.

For the last nine lessons the tool for "understand this text" was a chat model. A chat model is a writer. Ask a writer to tick six boxes and you get the boxes, plus a preamble, plus sometimes a seventh box it invented, in a format you now have to parse and validate. It works. It is also a bit like hiring a novelist to sort the mail.

This lesson is about the other tool.

What a System One model is

A **System One model** does not write. You give it one input (the **state** - here, a call transcript) and a list of typed **questions**, and it returns **a probability for every possible answer to every question**, in one pass. Nothing to parse and nothing to validate: the answer cannot be anything but one of the options you defined.

Jev is TypeSafe's System One model ([announcement](#), [docs](#)). The name comes from "System 1" thinking: the fast, intuitive judgement you make before you start reasoning. Three question types cover most decisions software makes:

Type	Asks	Answer
noul	a yes/no question	P(yes), a number from 0 to 1
choice	pick one of 2-255 labels	a probability per label, the top label, a confidence
score	a rating on 2-10 ordered levels	a probability per level, a weighted score, a confidence

The same call, two ways

This is call K-1004 from the lesson's data. The caller is very calm. The car is upside down.

Agent: Kestrel Mutual claims, Bea speaking. How can I help?
Caller: Good afternoon. I'd like to report a small incident, no rush.
Caller: My car rolled over on the ring road about ten minutes ago. I'm still sitting in it, upside down.
Caller: I think my arm is broken. It is pointing somewhere new.
Agent: Are you able to get out of the vehicle?
Caller: I'd rather not try, to be honest.

A local LLM asked to fill in the form as JSON (qwen3:1.7b, recorded) wrote:

```
{
  "line": "health",
  "intent": "new_claim",
  "severity": "major",
  "emergency": "no",
  "fraud_signals": "no",
  "needs_adjuster": "yes"
}
```

Valid JSON, every value a real option - and it took the caller at their word. "No rush", so `"emergency": "no"`; an arm at a new angle, so `"line": "health"`. The policy turns that into **assign adjuster**. There is no number in there to be suspicious of: `"no"` is just `"no"`.

The simulated Jev (qwen3.5:4b, recorded) answered the same six questions like this:

```
line          auto
  auto 1.00  home 0.00  health 0.00  travel 0.00  life 0.00
intent        new_claim
  new_claim 0.88  claim_status 0.02  coverage_question 0.06  complaint 0.01  cancel_policy 0.02
severity      major
  none 0.01  minor 0.01  moderate 0.02  major 0.88  catastrophic 0.09
```

```

emergency      yes
  yes 0.93 no 0.07
fraud_signals  no
  yes 0.21 no 0.79
needs_adjuster no
  yes 0.21 no 0.79
-> action: dispatch emergency help

```

Every question has every option, with a probability. emergency is not "yes", it is $P(\text{yes}) = 0.93$, and the policy compares that number with a threshold you chose.

Jev vs an LLM

	LLM (Lessons 1-9)	System One model (Jev)
Output	text, token by token	a probability per option, one pass
Typed?	only if you validate what it wrote	always: the options are the output
Probabilities	a chat reply carries none (Concept 3 digs them out, at a cost)	calibrated, per TypeSafe: 0.8 should be right 8 times in 10
Many questions	one long answer, or one call each	all of them in one request, per TypeSafe
Explains itself	yes	no
Writes, summarises, converses	yes	no
Speed (TypeSafe's figures)	seconds	70-500 ms
Price (TypeSafe's figures)	input and output tokens	input tokens only; the answers are free

The last two rows are TypeSafe's claims, not measurements made here. Everything measured in this lesson says so where it appears.

So why is it a good choice, and for what? Whenever the thing after the model is code. A routing table, a threshold, a queue, a fraud rule: they want a value from a known set and a number to compare, and a System One model gives exactly that, quickly and cheaply enough to run on every call. It is the wrong choice whenever a person has to read the output: it cannot write the reply to the caller, summarise the call, or explain itself. Most real systems want both: the judge and the writer.

What exists today (checked 2026-10-03)

What	Where it runs	Used in this lesson
TypeSafe Jev (jev-1.13.0, route jev-latest)	TypeSafe's servers only	yes: one real call (Step 1), more if you want
typesafe-sdk (PyPI, 0.7.2)	your machine, calls the API	yes, pinned with sha256 hashes

@typesafe-ai/sdk (npm, 0.6.0)	your machine, calls the API	yes, pinned, provenance-attested
A local Jev	does not exist	-
Community look-alikes on GitHub	your machine	no : unsigned, no releases

There is **no local Jev**. TypeSafe ships the model only as a hosted API. So the lesson has two halves:

- 1. **One real call** to Jev, so you have seen the actual thing answer.
- 2. **Everything else locally**, with a **Jev-like adapter**: a small server that speaks the same API and answers with a local model through Ollama. It is labelled **simulated** everywhere it appears. It teaches how the idea works and what it costs to fake - not how good Jev is.

A name to watch: `pip install qev` does **not** install anything from TypeSafe. That PyPI name belongs to an unrelated project. The official Python package is `typesafe-sdk`.

Before you start

You need	For	Get it
Nothing	<code>./run -l 10 demo</code> (recorded runs)	-
Ollama + <code>qwen3:1.7b</code> (1.4 GB)	the local session: the LLM arm and the simulated Jev	<code>ollama pull qwen3:1.7b</code>
<code>qwen3.5:4b</code> (3.4 GB), optional	the better, slower simulated Jev	<code>ollama pull qwen3.5:4b</code>
A TypeSafe account and API key	the one real call (<code>./run -l 10 hello</code>)	steps below

Getting a TypeSafe key:

- 1. Open the [TypeSafe Playground](#) and log in or create an account. You can try questions there in the browser before writing any code.
- 2. Create an API key on the [keys page](#).
- 3. Put it where the lesson finds it: `bash export TYPESAFE_API_KEY=... # this shell: works for every command echo 'TYPESAFE_API_KEY=...' >> .env # or the repo's .env (git-ignored): read by the # Python commands; Node.js and C# need the export`
- 4. Jev is in early access. If signing up puts you on a waitlist, carry on: every step of the lesson runs without a key, on the simulated Jev.
- 5. Worth a skim: the [quick start](#), the [API reference](#) and the [models page](#) (limits and prices: you pay for input tokens, not for answers).

Then let the lesson check your machine:

```
./run -l 10 check
```

```
Lesson 10 setup check (yes = ready, no = needed and missing, -- = optional)

yes Ollama running at http://127.0.0.1:11434          version 0.20.4
yes model qwen3:1.7b (needed)                        pulled
yes model qwen3.5:4b (optional, slower, better)      pulled
yes log-probabilities from the model                 20 candidates for the first token
-- TYPESAFE_API_KEY (for ./run -l 10 hello)          create one: https://console.typesafe.ai/keys
yes official Python SDK (for ./run -l 10 sdk)        installed in .venv-sdk
yes Node.js (for --lang node)                        found
-- .NET 8 SDK (for --lang csharp)                    https://dotnet.microsoft.com/download

Ready for the local session: ./run -l 10 live --backend keywords,llm-json,local
```

yes means ready, no means needed and missing (with the command that fixes it), -- means optional. It exits 1 only when the local session cannot run. No key? The lesson still works; Step 1 falls back to the simulated Jev and tells you so.

Step 1 · Hello, Jev: one real call

```
./run -l 10 hello
```

This sends **one** fake call (the calm caller in the upside-down car) to TypeSafe and prints the request, the response, the time it took, the tokens it used, and the action the lesson's policy takes. One request, six questions, a fraction of a cent. The text leaves your machine, which is why it is fake.

Without a key it prints the three steps to get one and sends the same request to the local adapter instead, clearly marked as **not** the real Jev. Here is that fallback, on this lesson's laptop:

```
One call to the Kestrel Mutual claims line (K-1004, fake):

Agent: Kestrel Mutual claims, Bea speaking. How can I help?
Caller: Good afternoon. I'd like to report a small incident, no rush.
Caller: My car rolled over on the ring road about ten minutes ago. I'm still sitting in it, upside down.
Caller: I think my arm is broken. It is pointing somewhere new.
Agent: Are you able to get out of the vehicle?
Caller: I'd rather not try, to be honest.

TYPESAFE_API_KEY is not set, so this is NOT the real Jev.
1. Log in or create an account: https://console.typesafe.ai/playground
2. Create an API key:          https://console.typesafe.ai/keys
3. export TYPESAFE_API_KEY=... and run ./run -l 10 hello again
Meanwhile: the same request to the local Jev-like adapter (qwen3:1.7b, simulated).

Request, abridged (local adapter, 6 Ollama calls): model, state, and 6 typed questions
(request JSON omitted here)

Response in 13.17s, 6 model call(s):
line          health
  auto 0.00  home 0.00  health 1.00  travel 0.00  life 0.00
intent        coverage_question
  new_claim 0.07  claim_status 0.00  coverage_question 0.93  complaint 0.00  cancel_policy 0.00
severity      minor
  none 0.00  minor 1.00  moderate 0.00  major 0.00  catastrophic 0.00
emergency     yes
  yes 1.00  no 0.00
fraud_signals no
  yes 0.00  no 1.00
needs_adjuster no
  yes 0.00  no 1.00
```

```
The policy (python/policy.py) turns that into: dispatch emergency help
No text was generated and nothing was parsed: every answer is one of your options,
with a probability your code can compare against a threshold.
```

Look at what came back: six answers, each one an option you defined, each with a probability. No JSON to repair. (Also look at **which** answers: the small local model thinks a rolled car is a minor health matter. Three of six are wrong. Hold that thought; the scorecard comes back to it.) That is the whole trick, and the rest of the lesson is about what you can build on it - and how far a local imitation gets.

Step 2 · The local session: three local engines, one scorecard

From here on nothing leaves your machine.

The data (all fake, all labelled)

`data/generate.py` writes two call centers from a fixed seed. Each record is a short transcript, the way speech-to-text hands a call to software, with the answers a careful human gave it:

File	Calls	Questions
<code>insurance.jsonl</code>	36 calls to the claims line of Kestrel Mutual	line, intent, severity, emergency, fraud_signals, needs_adjuster
<code>media.jsonl</code>	24 calls to the subscriber line of The Harbour Ledger , a newspaper	topic, churn_risk, wants_refund

The questions are in `data/questions.json`, written in TypeSafe's own request format.

About a third of the calls are **traps**, each marked with a `trap` field. The labels judge what happened, not how loudly it was said:

- the calm caller in the upside-down car (an emergency, reported as "a small incident")
- the furious caller whose premium went up (loud, and nothing is damaged)
- the "total loss" that turns out to be a meatball sandwich left on a dashboard in July
- polite callers whose stories do not add up: a theft days after the upgrade, a burglary with no broken window and no police report
- the caller who asks whether an ambulance is covered, because they need one, now
- callers in Spanish and German
- two callers who read out an instruction addressed to "the AI" - Lesson 4's prompt injection, now available by phone
- at the newspaper: thirty-one years a subscriber and **furious** about a paper that was twenty minutes late (not leaving), and a perfectly polite one who is (leaving)

Names come from a fixed list, emails are `@example.com`, phone numbers are `555-01xx` (reserved for fiction), policy numbers say `TEST`.

The decision

Answers are not actions. `python/policy.py` turns the six answers into one of six things the insurer actually does:

```
P(someone needs help now) >= EMERGENCY      -> dispatch emergency help
intent unclear, a complaint, or a cancellation -> human agent
a status or coverage question                 -> self-service answer
a new claim, P(fraud signals) >= SIU          -> special investigations
a new claim: small, documented, clean          -> fast-track payout
any other new claim                           -> assign adjuster
```

The two expensive mistakes pull in opposite directions: investigate an honest caller and you have insulted a customer; fail to investigate a staged accident and you have paid for it. `SIU` is the knob between them.

Four engines, one interface

Engine	What it is	Model calls per call
keywords	rules in <code>data/rules.json</code> , written from the question criteria before reading any call	0
LLM writes JSON	a local chat model (Ollama) asked to fill in the six fields as JSON	1
Jev-like adapter	this lesson's server: the same local model answering one lettered question at a time (simulated)	6
TypeSafe Jev	the real model, over the internet	1

All four produce a System One response, so the policy and the scorecard cannot tell them apart.

Run it

```
./run -l 10 demo          # Python: replay the recorded runs, print the scorecard
./run -l 10 --lang node demo # the same scorecard from Node.js
./run -l 10 --lang csharp demo # and from C#
./run -l 10 demo --dataset media # the newspaper instead of the insurer
./run -l 10                # the playground (needs Flask)
```

The one live run: the same question, two ways

The scorecard above is replayed from recordings. If you run one thing live, run this:

```
./run -l 10 race
./run -l 10 race "Caller: my car is on fire"    # or your own text
```

It asks **one** yes/no question - **does someone need help right now?** - about one call, of the **same local model**, twice: once as a chat model that writes its answer, once through the Jev-like adapter that lets it write a single token and reads the probability off it. With

`TYPESAFE_API_KEY` set, the real Jev joins as a third row. The playground (`./run -l 10`) has the same thing as a **Race** switch: both runs, their seconds, and how many times faster.

Call: Caller: A pipe has burst upstairs, water is coming through the kitchen ceiling and the light fitting is sparking. My mother is in there, she's 84.
Question: Does someone need help right now?

engine	answer	read	write	total	tokens
Ollama chat (qwen3:1.7b)	Yes. The person needs hel...	1.15s	2.31s	3.62s	40
Jev-like adapter (qwen3:1.7b, simulated)	P(yes) = 1.00	2.49s	0.00s	2.68s	1

What the chat model wrote:
Yes. The person needs help because the pipe has burst, water is coming through the kitchen ceiling, and there is a sparking light fitting, which could lead to a fire or electrical hazard.

Total: 3.62s vs 2.68s - the one-token answer is 1.4x faster here.
Writing the answer: 40 tokens took 2.31s; 1 token took 0.00s.
Reading the prompt: 1.15s (69 tokens) vs 2.49s (135 tokens).
The adapter's prompt also lists the options, so it reads more.
That is for ONE question. The adapter needs one model call per question; the real Jev answers all of a request's questions in one pass.
Add the real Jev to this table: `export TYPESAFE_API_KEY=...`
(create a key at <https://console.typesafe.ai/keys>)

Read the lines under the table, because the headline number is modest and the reason is the interesting part. The chat model spent 2.3 of its 3.6 seconds **writing** 40 tokens. The adapter spent nothing on writing: one token, then it read the probability off it. But its prompt is twice as long (it lists the options), and on a laptop CPU reading a prompt is slow too, so the total is only about 1.4 times faster.

Three honest conclusions:

- A decision does not need generated text, and not generating it is where the time goes away.
- A general chat model pressed into this job still pays for a long prompt, once per question. On the full six-question form that makes the adapter **slower** than one JSON reply (the scorecard's `s/call` column shows it).
- A model and a server built for this - one pass, many questions, no generation - is the part you cannot simulate on a laptop. TypeSafe quotes 70-500 ms for Jev; with a key, the third row of this table lets you check that claim yourself.

Scoring live, on your own machine, any mix of engines on the same calls:

```
./run -l 10 live --backend keywords,llm-json,local --limit 10
./run -l 10 live --backend keywords,llm-json,local,typesafe    # adds real Jev: 36 requests, fake data
```

Read the output

Recorded on 2026-10-03 on an Intel Core i5-10310U laptop (8 threads, 19 GB RAM, no GPU), Ollama 0.20.4, one model at a time. The full output is in `expected-output.txt` (insurance) and `expected-output-media.txt` (newspaper).

Kestrel Mutual claims line, 36 calls:

Engine	Right of 36 (line / intent / severity / emergency / fraud / adjuster)	Brier	Actions right	Suspicious claims missed	s / call
keywords (rules)	26 / 31 / 18 / 33 / 32 / 25	0.472	21/36	4 of 6	0
LLM writes JSON (qwen3:1.7b)	21 / 33 / 13 / 33 / 30 / 26	0.556	19/36	6 of 6	12.3
Jev-like adapter (qwen3:1.7b)	33 / 16 / 10 / 24 / 21 / 19	0.843	14/36	5 of 6	16.9
Jev-like adapter (qwen3.5:4b)	36 / 34 / 22 / 34 / 35 / 25	0.214	28/36	1 of 6	100.2

No engine sent an honest caller to investigations. TypeSafe Jev is not in the table: recording it needs an API key (`./run -l 10 record --backend typesafe`).

What the numbers say:

- **The format is not magic. The model has to be up to it.** The simulated Jev on the 1.7B model is the **worst** row: 14 of 36 actions, a Brier score of 0.843. On these longer transcripts the small model falls back on favourite letters - it rates 33 of the 36 calls "minor", including a motorway crash and a death claim. The same trick on the 4B model is the best row by a distance: 28 of 36, Brier 0.214. A System One answer is only as good as the model behind it, which is the whole case for a model trained to answer this way.
- **The LLM that writes JSON never once smelled a rat.** All six suspicious claims went through (6 of 6 missed), and it sent the calm caller in the upside-down car to an adjuster. It wrote perfectly valid JSON while doing so. Valid is not the same as right.
- **The caller who asked whether an ambulance is covered (K-1023)** got "self-service answer" from the JSON arm: it answered the question that was asked. Both adapters, and even the keyword rules, sent help.
- **Calibration is what makes the knob work.** Only the 4B adapter's answers move with SIU: no suspicious claim missed at 0.3, one at 0.6, three at 0.9 - and no honest caller investigated at any of them. The burglary with no broken window (K-1017) scores P(fraud) = 0.52: under the default 0.6, caught at 0.3. That is a business decision, expressed as a number.
- **Nobody is perfect.** The 4B adapter dispatched emergency help to a Spanish-speaking caller with a flooded kitchen and nobody hurt (P = 0.53, just over the line), and severity is hard for every engine (22 of 36 at best).
- **The prompt injection read out loud (K-1020)** fooled only the keyword rules here. Do not build a security model on that; build it on Lesson 4.
- **Cost.** Six questions are six model calls for the adapter: 100 seconds per call on the 4B model on this CPU. That is the price of faking it locally. Jev answers all six in one request;

TypeSafe quotes 70-500 ms.

The Harbour Ledger subscriber line, 24 calls - a useful dose of humility:

Engine	Right of 24 (topic / churn / refund)	Brier	Actions right	Retention desk: wrong / missed
keywords (rules)	20 / 18 / 21	0.361	17/24	2 / 2
LLM writes JSON (qwen3:1.7b)	16 / 3 / 18	0.956	16/24	5 / 1
Jev-like adapter (qwen3:1.7b)	20 / 3 / 23	0.658	13/24	7 / 1
Jev-like adapter (qwen3.5:4b)	22 / 8 / 22	0.360	12/24	10 / 0

Here the **keyword rules win** on actions. The models overrate churn: to a language model, almost anyone who phones a newspaper sounds like they are about to leave. The two small-model rows get churn right on 3 calls of 24, and rate even the reader of thirty-one years a medium or high risk. The 4B adapter knows that reader is staying, ties the rules on calibration (Brier 0.360) and misses no leaving reader, but it still sends ten happy ones to the retention desk. Raise `RETAIN` from 0.5 to 0.9 and that drops to four, with one missed - the knob again. And the JSON arm produced the lesson's only invalid answer: for the first call it wrote `"topic": "subscriber services"`, a seventh box it invented from the agent's greeting.

Two lessons in one table: sometimes rules are enough, and "churn risk" needs a better question than the one written here. Exercise 3 is yours.

Concept 1 · The whole API is one POST

`python/systemone.py` builds the request, sends it with the standard library, and reads the answer:

```
POST https://api.typesafe.ai/v1/systemone
Authorization: Bearer <TYPESAFE_API_KEY>
{"model": "jev-latest", "state": "Agent: ...\\nCaller: ...", "questions": {"intent": {...}, "severity": {...}}
```

`read_answers()` turns every answer into the same shape - `{"pick", "probs", "confidence"}` with labels, not wire keys - and reports any answer that does not match the question it was asked. With a real System One model that list stays empty. It is there because this lesson also scores engines that can return anything.

`post()` refuses to send anything anywhere except `https://api.typesafe.ai` or a loopback address. A typo in `TYPESAFE_BASE_URL` cannot hand your key, or your callers, to a stranger.

Concept 2 · The official SDK - the same code for Jev and for the local adapter

```
./run -l 10 install-sdk          # once: typesafe-sdk 0.7.2 into lessons/10-*/.venv-sdk
./run -l 10 sdk "Caller: someone dented my parked car, I have a photo"
```

python/sdk_example.py is the whole integration:

```
client = TypeSafeClient()          # reads TYPESAFE_API_KEY and TYPESAFE_BASE_URL
result = client.system_one(text, {
    "intent": Choice(instructions="What does the caller want from this call?",
        criteria={"new_claim": "Reports a loss or sends in a bill to be paid", ...}),
    "severity": Score(instructions="How serious is the damage, loss or harm described?",
        criteria=["none", "minor", "moderate", "major", "catastrophic"]),
    "fraud_signals": Noul(instructions="Does the story have warning signs of a dishonest claim?"),
})
result.choices["intent"].probabilities    # {"new_claim": 0.91, "claim_status": 0.03, ...}
```

Node.js (node/sdk_example.mjs):

```
import { TypeSafeClient, choice, noul, score } from "@typesafe-ai/sdk";
const client = new TypeSafeClient();
const { answers } = await client.systemOne({ state: text, questions: {
    intent: choice("What does the caller want from this call?",
        { new_claim: "Reports a loss or sends in a bill to be paid", /* ... */ }),
    severity: score("How serious is the damage, loss or harm described?",
        ["none", "minor", "moderate", "major", "catastrophic"]),
    fraud_signals: noul("Does the story have warning signs of a dishonest claim?"),
}});
```

C# has no official SDK, so dotnet/SystemOne.cs posts the same JSON with HttpClient. Point any of the three at the local adapter instead of TypeSafe by changing **one environment variable**:

```
./run -l 10 serve                # terminal 1: the adapter on 127.0.0.1:8765
export TYPESAFE_BASE_URL=http://127.0.0.1:8765    # terminal 2
export TYPESAFE_API_KEY=local          # the adapter ignores it; the SDK wants one
./run -l 10 sdk "Caller: my car was stolen the day after I upgraded my cover"
```

How the SDKs were installed - and why that matters

Both SDKs are pinned, so what runs is what was reviewed:

- **Python:** requirements-sdk.txt lists every package with its sha256. pip install --require-hashes refuses a file whose hash differs. Change one hash and the install stops with THESE PACKAGES DO NOT MATCH THE HASHES.
- **Node.js:** node/package.json pins @typesafe-ai/sdk to an exact version, and package-lock.json records its integrity hash. npm audit signatures checks the registry signature and the provenance attestation: npm can show this package was built from github.com/typesafe-ai/typesafe-sdk-js by GitHub Actions.

Neither is needed for the demo or the tests. The SDK test is skipped when the SDK is absent.

Concept 3 · Making a local LLM answer like a System One model

`python/local_adapter.py` is a small server with the same endpoint. For each question it:

1. Writes a multiple-choice prompt - options lettered A, B, C... - with the transcript fenced as data.
2. Lets the model produce **one token**, at temperature 0.
3. Reads Ollama's **log-probabilities** for the top 20 candidates of that token, and gives each option the probability of its letter, renormalised.

That is a real probability for every option, from one short forward pass: the System One contract, built from parts you already had. What it does not have is Jev's training for the job:

- **Calibration.** A small model is often 100% sure and wrong. The Brier score shows it.
- **Cost.** One model call per question. Jev answers all the questions of a request in one pass.
- **Speed.** Seconds on a laptop CPU, per question.

The server binds to `127.0.0.1` and refuses anything else: it has no authentication.

Concept 4 · The model answers; your policy decides

`python/policy.py` has four thresholds - `EMERGENCY`, `SIU`, `FAST_TRACK`, `CONFIDENT` - and no model. This is the most important file in the lesson:

- The model never says "send this to investigations". It says $P(\text{fraud signals}) = 0.83$. Your code decides that 0.83 is enough.
- Moving a threshold changes behaviour **without retraining or re-prompting anything**. The demo prints the investigations at $SIU = 0.3, 0.6$ and 0.9 .
- That only works if the probabilities mean something. Rules and JSON answers are always 0 or 1, so for them the knob is decorative.

Concept 5 · Scoring a decision engine

`python/scorecard.py`:

Metric	Answers the question
accuracy	how often is the top answer right?
typed	how often was the answer a valid option at all?
Brier score	when it says 0.9, is it right 9 times in 10? 0 is perfect; certain and wrong costs 2
actions	does the policy do the same thing it would do with the human labels?

wrong / missed	honest callers investigated / suspicious claims not investigated
----------------	--

Accuracy alone would rank an engine that is confidently wrong next to one that knows when it is unsure. Only the second one lets a threshold do its job. Lesson 5 scored a RAG pipeline against a golden set; this is the same discipline for decisions.

Concept 6 · Recorded once, replayed everywhere

`./run -l 10 record --backend local` asks the engine about every call and saves the replies in `data/cassettes/`, with the date, the hardware and a digest of each request. `demo` replays them through the same parsing, policy and scorecard code, in all three languages. If a call or a question changes, its digest stops matching and the replay refuses instead of scoring a conversation that never happened (the Lesson 9 cassette idea). The prompts have fingerprints too: change one and its recordings must be redone.

How-to

Run the local adapter and call it

```
ollama pull qwen3:1.7b
./run -l 10 serve --model qwen3:1.7b
# another terminal
curl -s http://127.0.0.1:8765/v1/systemone -H 'Content-Type: application/json' -d '{
  "model": "local", "state": "Caller: my kitchen is under water",
  "questions": {"emergency": {"type": "noul", "instructions": "Does someone need help right now?"}}}'
```

Ask about one call, with any engine

```
./run -l 10 ask "Caller: I hit a deer. The car still drives, the headlight is gone." --backend local
./run -l 10 ask "Caller: Cancel everything unless it arrives before seven." --dataset media --backend local
./run -l 10 ask "Caller: I hit a deer." --backend keywords
```

Score engines on your machine - local and TypeSafe side by side

```
./run -l 10 live --backend keywords,llm-json,local --limit 10
./run -l 10 live --backend local --model qwen3.5:4b --dataset media
./run -l 10 live --backend keywords,llm-json,local,typesafe # needs TYPESAFE_API_KEY
```

`--backend` takes one engine or several, comma-separated. Every engine answers the same calls in the same run and lands in one scorecard. An engine that cannot run - no API key, Ollama not started - gets a row saying why, and the others still run. With `typesafe`, the fake calls leave your machine. The playground (`./run -l 10`) has a live switch for each: the local adapter and TypeSafe.

Record what the demo replays

```
./run -l 10 record --backend local --model qwen3:1.7b --hardware "my laptop"
./run -l 10 record --backend typesafe # needs TYPESAFE_API_KEY
```

Recordings are named per engine, model and dataset (`jev-like-qwen3-1.7b-insurance.json`), so recording one never overwrites another. A run with `--limit N` writes `...-firstN.json`, which the demo does not read.

Record one model at a time with nothing else on the CPU. Two resident models on a 19 GB laptop got the Ollama service killed in Lesson 9.

Use your own call center

1. Write records as JSON lines with `id`, `text`, `labels` (one label per question) and `trap` ("" for none).
2. Add the questions to `data/questions.json`, keyword rules to `data/rules.json`, and a title and short column names to `data/datasets.json`.
3. Add a policy to `python/policy.py` (POLICIES), then `./run -l 10 live --backend local --dataset <name>`.

When to use which

Use	When	Call-center examples
Rules	the signal is a word or a number, and a miss is cheap	"claim number" -> status; caller is in the VIP list
System One model	a judgement over unstructured text, many items, typed output, a threshold	routing, fraud flags, emergency detection, churn risk, quality scoring of every call
LLM	the output is text, or the steps are not known in advance	the reply to the caller, the call summary for the adjuster, a tool-using agent (Lesson 9)
Both	an LLM acts, a System One model checks	Jev decides whether the LLM's drafted reply promises a payout it should not; Jev as the guard on Lesson 9's tool calls and Lesson 4's retrieved passages

Ideal applications share three properties: the answer is one of a known set, there are many items (every call, every claim, every row of a dataset), and a business rule needs a probability to compare against a threshold. TypeSafe's own list of [example use cases](#) names customer support and insurance claims among them.

Poor fits: anything that must be written, anything that needs reasons a person can read, and anything a keyword already decides.

Python, Node and C

	Python	Node.js	C#
--	--------	---------	----

demo (replay)	python/jev.py	node/jev.mjs	dotnet/Program.cs
live call	python/systemone.py, official SDK	official SDK	HttpClient
setup check, hello, live scoring, adapter server	python/	-	-
tests	python/test_jev.py	demo byte-diff	demo byte-diff

All three print the same scorecard, byte for byte.

Exercises

1. In the playground, pick K-1017 (the tidy burglars) on the 4B adapter and drag SIU down until the claim goes to investigations. Then look at the demo's SIU table: what does that setting do to the other 35 calls?
2. Add a question language (choice: en, es, de) to the insurance set and record it. Which engine handles it best?
3. The newspaper's churn_risk question makes every model nervous. Rewrite its instructions and criteria in data/questions.json, record again, and see whether the retention desk calms down.
4. Fix the keyword rules so the sandwich (K-1006) is no longer catastrophic. What did the fix break?
5. With a TypeSafe key: ./run -l 10 record --backend typesafe, then compare the real Jev's Brier score with the simulated one's.
6. Write a call that fools the 4B adapter. (The burglary in K-1017 slipped past it at the default threshold.)

From demo to production

- **Keep the policy in code, versioned and tested.** Thresholds are business decisions with an owner, not prompt wording.
- **Watch calibration over time.** Re-score a labelled sample each week; a Brier score that drifts up is the early warning.
- **Know where the data goes.** Real call transcripts are personal data. A hosted model means they leave your network; decide that on purpose.
- **Log the probabilities, not only the action.** They are what you will tune against later.
- **Treat the caller as untrusted input.** People will read instructions to your model down the phone. The model should never be the only thing between a sentence and a payout.
- **Emergencies do not wait for a model.** A real claims line routes "someone is hurt" to a human first and scores the call afterwards.

Next lesson

Lesson 11 · Microsoft Semantic Kernel (C#) → - the Lesson 9 agent rebuilt in .NET, where automatic function calling runs the loop for you.